

Navegação

Ir de uma tela a outra, e voltar trazendo o resultado

Prof. Cristiano Rodrigues

**Você aperta o botão voltar
sem pensar. Quem é que
lembra de onde você veio?**

Levantem hipóteses antes de eu mostrar.



Para acompanhar hoje

Você precisa saber

- Compor uma tela com Column, Row e Expanded, usando as cores do tema
- Guardar e atualizar estado dentro de uma tela com setState
- Esperar um resultado que demora, com async e await

Você precisa ter feito

- A lista de hábitos com o formulário na mesma tela, da prática anterior
- O projeto rodando, com o hábito entrando na lista ao salvar

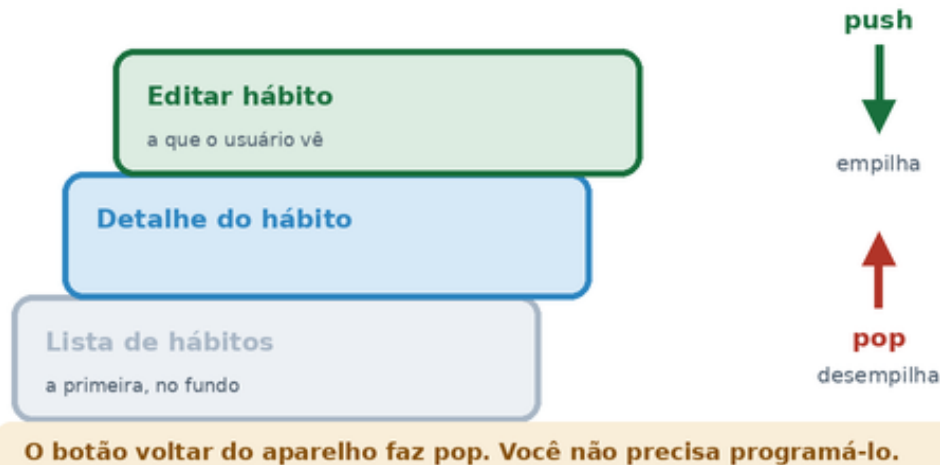
PARTE 01

A pilha de telas

Uma estrutura de dados que você já conhece, agora governando o aplicativo.

Quem lembra de onde você veio

A navegação é uma pilha de telas



A tela visível é sempre a do topo.

Ir para outra tela

```
onPressed: () {  
  Navigator.push(  
    context,  
    MaterialPageRoute(  
      builder: (context) => const TelaNovoHabito(),  
    ),  
  );  
},
```

lib/tela_habitos.dart · a tela nova entra deslizando, já com a seta de voltar

O botão voltar você ganha de graça.

A barra superior da tela empilhada já vem com a seta, e o botão do aparelho já faz o mesmo. Não programe nenhum dos dois. Só chame pop você mesmo quando o retorno acontecer por outro caminho, como um botão Salvar.



Três verbos sobre a mesma pilha

push

Põe uma tela em cima. A de baixo continua lá, intacta.

pop

Tira a do topo. É o que a seta de voltar já faz sozinha.

pushReplacement

Troca a do topo. A anterior sai da pilha e não volta.

pushReplacement é para quando voltar não faria sentido: da tela de login para a tela inicial.

PARTE 02

Levar e trazer dados

A tela de destino precisa saber o que mostrar, e às vezes precisa responder.

Levar: o dado vai no construtor

```
class TelaDetalhe extends StatelessWidget {  
  const TelaDetalhe({super.key, required this.habito});  
  final Habito habito;          // chega de fora, e não muda  
  
  @override  
  Widget build(BuildContext context) =>  
    Scaffold(appBar: AppBar(title: Text(habito.nome)));  
}  
  
// na tela da lista, ao tocar no item:  
MaterialPageRoute(builder: (_) => TelaDetalhe(habito: h))
```

lib/tela_detalhe.dart · a barra superior mostra o nome do hábito que foi tocado

Trazer o resultado de volta

Ir até a outra tela, e voltar trazendo o resultado



O await fica esperando. Ele só termina quando a outra tela sai, e devolve o que o pop entregou.

Quem navega espera; quem foi navegado responde.

Trazer: await no push, valor no pop

```
// na tela da lista
onPressed: () async {
  final novo = await Navigator.push<Habito>(
    context,
    MaterialPageRoute(builder: (_) => const TelaNovoHabito()),
  );
  if (!mounted) return;          // a tela pode ter saído
  if (novo != null) setState(() => _habitos.add(novo));
},

// na tela do formulário, no botão Salvar
Navigator.pop(context, habito);
```

lib/tela_habitos.dart · ao salvar, o hábito aparece na lista; ao voltar, nada muda

Depois do await, a tela pode não estar mais lá.

Enquanto a outra tela está aberta, o usuário pode fechar a que chamou. Quando o await termina, chamar setState numa tela que saiu dá erro. A proteção é uma linha: se a tela não estiver mais montada, não faça nada.



PARTE 03

Quando as telas viram muitas

Com quatro ou mais telas, espalhar a construção delas pelo código cobra caro.

Duas formas de dizer para onde ir

Direto

- Quem chama constrói a tela de destino, com push.
- Passar objetos é natural, pelo construtor.
- Com muitas telas, a construção se espalha.

Nomeada

- As telas ficam declaradas num lugar só, e usa pushNamed.
- Quem chama só precisa saber o nome da rota.
- Passar objetos exige mais cerimônia.

As telas declaradas num lugar só

```
MaterialApp(  
  initialRoute: '/',  
  routes: {  
    '/': (context) => const TelaHabitos(),  
    '/novo': (context) => const TelaNovoHabito(),  
    '/resumo': (context) => const TelaResumo(),  
    '/perfil': (context) => const TelaPerfil(),  
  },  
)  
  
// de qualquer lugar do aplicativo:  
Navigator.pushNamed(context, '/novo');
```

lib/main.dart · o aplicativo abre na lista, e pushNamed chega a qualquer tela do mapa

A tela que o mapa não consegue construir

```
routes: {  
  '/': (context) => const TelaHabitos(),          // sem dado  
  '/novo': (context) => const TelaNovoHabito(),    // sem dado  
  
  // e a tela de detalhe, que exige um hábito?  
  '/detalhe': (context) => TelaDetalhe(habito: ???),  
}  
  
// o mapa é escrito uma vez, quando o aplicativo sobe.  
// o hábito só existe depois, quando o usuário toca num item.
```

lib/main.dart · o mapa constrói a tela sem saber do que ela precisa

O critério é o dado, não o gosto

Nomeada

As telas do menu. Qualquer um abre, e elas não dependem de nada.

Direto

As telas que dependem de um item. O dado vai no construtor.

Misturar

É o normal. Todo projeto usa as duas, e escolher assim é maturidade.

Tela que precisa de um dado não entra no mapa: quem chama tem o dado, o mapa não tem.

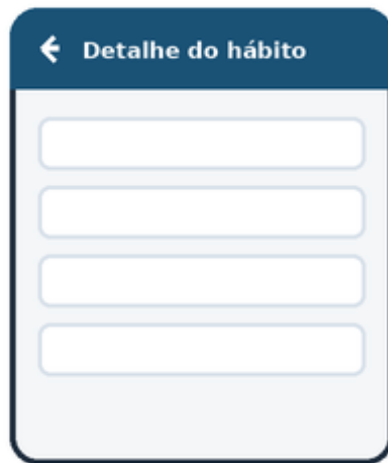
PARTE 04

Trocar de seção, sem empilhar

Nem todo caminho entre telas é ida e volta. Menu é outra coisa.

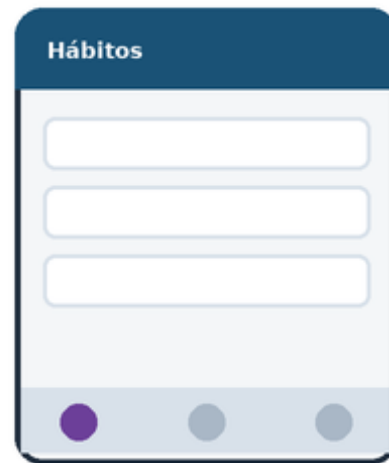
Empilhar e trocar não são a mesma coisa

Empilhar: push



Tem seta de voltar: a anterior continua embaixo.

Trocar: abas



Sem seta: trocar de aba não é voltar.

A seta de voltar é a pista visível de qual dos dois está acontecendo.

O esqueleto: um índice decide o que aparece

```
class _TelaPrincipalState extends State<TelaPrincipal> {  
  int _aba = 0; // qual seção está aberta  
  
  final _telas = const [  
    TelaHabitos(), TelaResumo(), TelaPerfil(),  
  ];  
  
  @override  
  Widget build(BuildContext context) => Scaffold(  
    body: _telas[_aba], // o corpo troca  
    bottomNavigationBar: _barra(), // a barra fica  
  );  
}
```

lib/tela_principal.dart · a barra fica fixa embaixo e o corpo muda ao tocar nela

A barra: um destino para cada tela

```
Widget _barra() => NavigationBar(  
  selectedIndex: _aba,  
  onDestinationSelected: (i) => setState(() => _aba = i),  
  destinations: const [  
    NavigationDestination(icon: Icon(Icons.list),      label: 'Hábitos'),  
    NavigationDestination(icon: Icon(Icons.bar_chart), label: 'Resumo'),  
    NavigationDestination(icon: Icon(Icons.person),    label: 'Perfil'),  
  ],  
);
```

lib/tela_principal.dart · três telas na lista, três destinos na barra

Menu não empilha.

Use push para ir a uma tela e voltar: um detalhe, um formulário, uma confirmação. Use a barra inferior para alternar entre as seções principais. Trocar de seção com push produz uma pilha que só cresce, e um botão voltar que ninguém entende.

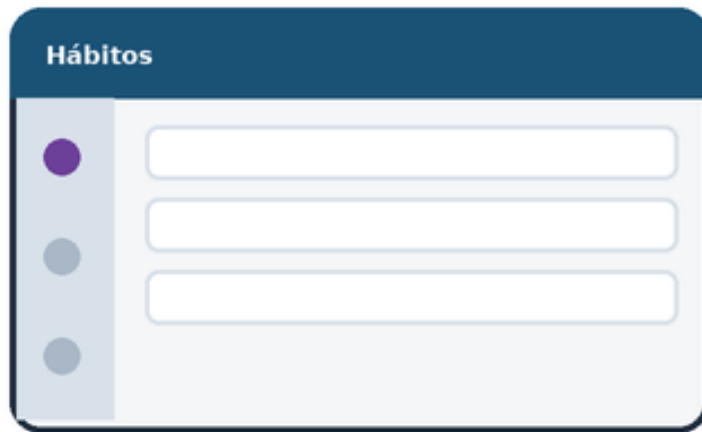


A mesma navegação, em telas de tamanhos diferentes

O mesmo aplicativo, em duas larguras



Tela estreita: barra embaixo



Tela larga: trilho na lateral

Barra embaixo no celular, trilho na lateral quando há largura sobrando.

PARTE 05

A prática de hoje

Separar o que está junto, e depois chegar às quatro telas da entrega.

Separe o formulário em uma tela própria



O mesmo formulário da prática anterior, agora em duas telas.

- **1. Mova o Form** para uma tela nova, com Scaffold.
- **2. Um botão de ação** na lista chama o push.
- **3. Espere com await** e trate o null.
- **4. No Salvar**, chame pop com o hábito.

Onde cada coisa mora depois da separação

tela_habitos.dart

A lista, o botão de ação e o método que abre a outra tela.

tela_novo_habito.dart

O Form que saiu da lista, com Scaffold próprio e o pop.

habito.dart

A classe Habito, usada pelas duas telas.

Uma tela por arquivo. Se a classe Habito ainda está no meio do código, mova.

Tudo isto está no catálogo, no Canvas

Navegação entre telas

Ir para outra tela · Levar um dado · Voltar com resultado
Rotas nomeadas · Dado por rota nomeada · Trocar a tela do topo

Navegação estrutural

Menu lateral · Barra inferior · Abas
Botão de ação principal · Barra superior com ações

Catálogo de componentes: o código mínimo, para que serve e a armadilha de cada um.

O que a Sprint pede, e o que você já tem

Já tem

Ir e voltar

push para abrir, pop para fechar, e o dado voltando junto.

Já tem

Menu entre seções

A barra inferior alternando as telas principais.

Falta juntar

Quatro telas com design

Layout e tema já são seus. Hoje as telas ganham ligação.

Saia daqui com as quatro telas navegáveis. O acabamento cabe no tempo que sobra.

Nesta aula, você aprendeu a:

- Empilhar, desempilhar e substituir telas: push, pop e pushReplacement
- Levar um dado para a tela seguinte pelo construtor
- Trazer um resultado de volta, com await no push e valor no pop
- Declarar rotas nomeadas e decidir quando elas compensam
- Trocar de seção com uma barra inferior, sem empilhar

Referências

Flutter. Navigation and routing. Disponível em: docs.flutter.dev/ui/navigation.

Flutter. Return data from a screen. Disponível em: docs.flutter.dev/cookbook/navigation/returning-data.

Flutter. Adaptive and responsive design. Disponível em: docs.flutter.dev/ui/adaptive-responsive.

Material Design 3. Navigation bar. Disponível em: m3.material.io/components/navigation-bar.

Catálogo de componentes de interface. Deck de referência da disciplina, no Canvas.